



Sapiientia Erdélyi Magyar Tudományegyetem
Marosvásárhelyi Kar

Felvételi, 2018. július
– Alapképzés, informatika vizsga –

JAVÍTÓKULCS

Hivatalból 1 pont jár. Munkaidő 3 óra.

Megjegyzések:

- A pszeudokód algoritmusban
 - az „=” értékadást jelent
 - Az $x[i][j]$ hivatkozás az x tömb i -edik sorának j -edik elemét jelenti.
 - A „**minden** $i = 1, n, 1$ **végezd**” jelentése: minden $i = 1, 2, 3, \dots, n$ értékre végezd el...
 - A „**minden** $i = n, 1, -1$ **végezd**” jelentése: minden $i = n, n-1, n-2, \dots, 1$ értékre végezd el...
- Minden egész érték tárolható `integer/int` típusú változóiban.
- A bemeneti adatok helyesnek tekinthetők.
- A programokat megjegyzésekkel kell ellátni!
- Törekedjünk hatékony megoldásokra!

1. (1 pont) Mit ír ki az alábbi pszeudokód algoritmus, ha az $a[1..5]$ öt-elemű tömb az 7, 3, 2, 9, 19 értékeket tárolja? Adjunk egy-egy példát olyan öt-elemű tömbökre, amelyekre az algoritmus IGEN-t, illetve NEM-et ír ki!

```

i = 1
j = 5
ha a[i] > a[i+1] ÉS a[j] > a[j-1] akkor
    i = i + 1
    j = j - 1
    ha a[i] > a[i+1] ÉS a[j] > a[j-1] akkor
        kiír "IGEN"
    különben
        kiír "NEM"
    ■
különben
    kiír "NEM"
■

```

IGEN

3, 2, 1, 2, 3
1, 2, 3, 4, 5

2. (1 pont) Mi lesz az $a[1..5]$ és $b[1..5]$ öt-elemű tömbök tartalma az alábbi pszeudokód algoritmusrészlet nyomán, ha előzőleg az a tömb a 3, 2, 1, 5, 4, a b pedig a 0, 0, 0, 0, 0 értékeket tárolta?

```

minden i = 1,5,1 végezd
    b[a[i]] = b[a[i]] + 1
    ■
minden i = 5,1,-1 végezd
    a[b[i]] = a[b[i]] - 1
    ■

```

-2, 2, 1, 5, 4
1, 1, 1, 1, 1

3. (1 pont) Mi lesz az $a[1..n][1..n]$ kétdimenziós tömb tartalma az alábbi pszeudokód algoritmusrészlet nyomán, ha $n=5$, és előzőleg minden eleme 0-t tárolt?

```

k = 1
minden i = n,2,-1 végezd
    minden j = 1,i-1,1 végezd
        a[i][j] = k
        a[n+1-i][n+1-j] = a[i][j]
        k = k + 1
    ■
■

```

0	4	3	2	1
10	0	7	6	5
8	9	0	9	8
5	6	7	0	10
1	2	3	4	0

4. (1 pont) Írjunk Pascal vagy C/C++ programot, amely kiírja, egymás alá a képernyőre, Mátyás király (Kolozsvár, 1443. február 23. – Bécs, 1490. április 6.) évenkénti születésnapjai dátumait "év. hó. nap." alakban (például az első születésnapja dátuma 1443. 02. 23. volt, az utolsóé pedig 1490. 02. 23.).

```
#include <stdio.h>

int main()
{
    int i;
    for ( i = 1443 ; i <= 1490 ; ++i ){
        printf("%i. 02. 23.\n", i);
    }
    return 0;
}
```

5. (1 pont) Írjunk Pascal vagy C/C++ programot, amely beolvas a billentyűzetről két természetes számot (*a* és *b*), majd kiírja, egymás alá a képernyőre, az évszámokat i.e. *a*-tól, egészen i.sz. *b*-ig. Az időszámításunk előtti évszámok előtt szerepeljen az "i.e.", az időszámításunk szerintiék előtt pedig az "i.sz." karakterlánc. (Vegyük figyelembe, hogy időszámításunk előtt, az idő múlásával csökkennek az évszámok, és nincs nulla év)

```
#include <stdio.h>

int main()
{
    unsigned a, b, i;
    scanf("%u%u", &a, &b);
    for ( i = a ; i > 0 ; --i ){
        printf("i.e. %u\n", i);
    }
    for ( i = 1 ; i <= b ; ++i ){
        printf("i.sz. %u\n", i);
    }
    return 0;
}
```

6. (1 pont) Írjunk Pascal vagy C/C++ függvényt, amely egy számsorozatról eldönti, hogy palindrom-e (visszafelé olvasva is ugyanaz). A függvény paraméterei legyenek a számsorozatot tároló tömb, illetve az elemek száma, és térítse vissza a 0 vagy 1 értéket (attól függően, hogy a válasz NEM vagy IGEN).

```
int palindrom(int *a, int n){  
    int i, j;  
    for ( i = 0, j = n-1 ; i < j ; ++i, --j ){  
        if ( a[i] != a[j] ){  
            return 0;  
        }  
    }  
    return 1;  
}
```

7. (1 pont) Egy bemeneti állományban adott az n ($2 \leq n \leq 99$) érték és egy $n \times n$ méretű természetes számokat tartalmazó mátrix, amelynek minden sora egy-egy növekvő számsorozat. Írjunk Pascal vagy C/C++ programot, amely beolvassa a mátrixot, és megkeresi azt az n elemű, szigorúan növekvő számsorozatot, amelynek i -edik eleme a mátrix i -edik sorából származik, és legnagyobb az elemeinek összege. A képernyőre csak a maximális összegértéket kell kiírni. Ha nem létezik egyetlen számsorozat sem, amely megfelelne a követelményeknek, akkor nullát írjon ki a program.

Példa:

BEMENET	KIMENET
3	19
1 4 6	
3 5 6	
2 2 9	

```
#include <stdio.h>

int legnagyobbnalakisebb(int a[100][100], int, int, int);

int main()
{
    int n, a[100][100], i, j;
    freopen("input7.txt", "rt", stdin);
    scanf("%i", &n);
    for ( i = 0 ; i < n ; ++i ){
        for ( j = 0 ; j < n ; ++j ){
            scanf("%i", &a[i][j]);
        }
    }

    //ÖTLET: az utolsó sorból az utolsó elemet válasszuk (a[n-1][n-1])
    //majd minden i-edik sorban (i=n-2,n-3,...0) megkeressük,
    //amennyiben létezik ilyen,
    //a legnagyobb olyan elemet, amely kisebb,
    //mint az (i+1)-edik sorból választott

    int utolso, utolsoelotti, osszeg;
    utolso = osszeg = a[n-1][n-1];
    for ( i = n-2 ; i >= 0 ; --i ){
        utolsoelotti = legnagyobbnalakisebb(a,i,n,utolso);
        if ( utolsoelotti == -1 ){
            break;
        }
        utolso = utolsoelotti;
        osszeg += utolso;
    }
```

```
        if ( i < 0 ){
            printf("%i", osszeg);
        }
        else{
            printf("0");
        }
        return 0;
    }
```

```
int legnagyobbnalakisebb(int a[][100], int i, int n, int utolso){
//Visszatéríti az a tömb i-edik sorából a legnagyobb olyan elemet,
//amely kisebb, mint az utolso paraméter
//Ha nincs ilyen elem, akkor (-1)-et térít vissza
    int j = n-1;
    while ( j >= 0 && a[i][j] >= utolso ){
        --j;
    }
    if ( j < 0 ){
        return -1;
    }
    else{
        return a[i][j];
    }
}
```

8. (1 pont) Egy bemeneti állományban adott az n ($2 \leq n \leq 99$) érték és egy $n \times n$ méretű természetes számokat tartalmazó mátrix (az állomány első sora tartalmazza az n értéket, majd a következő n sor a mátrix sorainak az elemeit, egy-egy szóközzel elválasztva). Írjunk Pascal vagy C/C++ programot, amely beolvassa a mátrixot, majd eldönti, hogy keretről keretre, befelé haladva a keretek elemeinek összegértékei csökkenő számsorozatot képeznek-e vagy sem. A példa bemenetre azért kerül az IGEN szó kiírásra (a képernyőre), mert a külső kereten az összeg 30, a következőn 24, a legbelsőn pedig 18 (csökkenő számsorozat).

Példa:

BEMENET	KIMENET
6	IGEN
1 2 1 2 1 2	
2 2 2 2 2 1	
1 2 4 5 2 2	
2 2 5 4 2 1	
1 2 2 2 2 2	
2 1 2 1 2 1	

```
#include <stdio.h>

int main()
{
    int n, a[100][100], b[100] = {0}, i, j, k;
    freopen("input8.txt", "rt", stdin);
    scanf("%i", &n);
    for ( i = 0 ; i < n ; ++i ){
        for ( j = 0 ; j < n ; ++j ){
            scanf("%i", &a[i][j]);
        }
    }

    for ( k = 0 ; k <= (n-1)/2 ; ++k ){
        //keretről keretre haladunk, befelé
        if ( n % 2 && k == (n-1)/2 ){
            //ha n páratlan, és a legbelső, egy-elemű kereten vagyunk
            b[k] = a[k][k];
        }
        else{
            //a k. keret négy sarka
            b[k] += a[k][k] + a[n-1-k][k] + a[n-1-k][n-1-k] + a[k][n-1-k];
        }
    }
```

```

        for ( i = k, j = k+1 ; j < n-1-k ; ++j ){
            //a k. keret felső oldalának sarkok közé eső szakasza
            //a kurrens elem (a[i][j]) mellett, ennek főátlóra, mellékátlóra és
            //középpontra való szimmetrikusait is tekintjük
            b[k] += (a[i][j] + a[j][i] + a[n-1-i][n-1-j] + a[n-1-j][n-1-i]);
        }
        printf("%i\n", b[k]);
        if ( k > 0 && b[k] > b[k-1] ){
            //ha elromlik a csökkenő sorrend, leállítjuk a ciklust
            break;
        }
    }
    if ( k > (n-1)/2 ){
        printf("IGEN");
    }
    else{
        printf("NEM");
    }
    return 0;
}

```


9. (1 pont) Tekintsünk n téglalapot ($1 \leq n \leq 999$). A téglalapok magassága azonos, szélességük pedig rendre $a_1, a_2, \dots, a_n$. Írjunk Pascal vagy C/C++ programot, amely beolvassa állományból az n értékét, majd az n darab szélesség-értéket (egész számok), és kiírja a képernyőre azt, hogy maximum hány szintes piramis építhető a téglalapokból úgy, hogy minden szintre (kivéve a legalsót) igaz legyen az, hogy

- szélessége kisebb, mint a közvetlen alatta levő szinté,
- kevesebb téglalapot tartalmaz, mint a közvetlen alatta levő szint.

(Az ábra egy lehetséges kétszintes elhelyezést mutat be a példa bementre)

Példa:

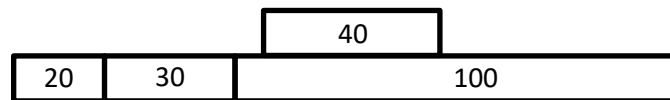
BEMENET

KIMENET

4

2

40 100 20 30



```
#include <stdio.h>
#include <math.h>
int main()
{
    //ÖTLET:
    //1. Mindig tekinthetjük a téglalapokat szélességük szerint
    //növekvő sorrendben
    //2. Fentről lefele építjük a piramist
    //3. A maximális magasság elérése érdekében a legfelső szintre
    //egy téglalapot teszünk, majd minden következő szintre eggyel többet
    //(vegyük észre, hogy automatikusan teljesül az a feltétel is,
    //hogy a szintek szélessége tekintetében is meglegyen a piramis jelleg)
    //4. A fentiekkel összhangban a maximális szint az a legnagyobb k érték
    //lesz, amelyre a  $1+2+\dots+k = k*(k+1)/2$  érték még kisebb vagy egyenlő,
    //mint n

    int n, maxszint;
    freopen("input9.txt", "rt", stdin);
    scanf("%i", &n);
    maxszint = floor((-1+sqrt(1+(8*n)))/2);
    printf("%i", maxszint);
    return 0;
}
```